MINI-PROJET POO · SUJET 9

Billetterie d'un festival : Rivages d'Été

Cahier des charges pédagogique

Wissem Zeddini · À lire avec le document « Mini-projets POO : cadre commun »

Client	M. Fares Kammoun, responsable de la billetterie du festival Rivages d'Été (client simulé)
Le logiciel	La billetterie d'un festival de musique d'été (fictif) : billets de plusieurs types, quotas par catégorie, annulations et recettes
Difficulté	Intermédiaire (2 sur 3), avec un défi métier sur les quotas et les calculs en entiers
Notions du socle	Encapsulation, constructeurs, héritage, redéfinition, polymorphisme
Bonus possibles	Abstraction (badge Abstrait), exception personnalisée (badge Résilient), sauvegarde dans un fichier (badge Archiviste)
Livrable final	Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes

1. Le contexte

Le festival **Rivages d'Été** (un festival fictif) accueille chaque année quelques milliers de spectateurs pendant trois soirées de musique, au bord de la mer. Jusqu'ici, les billets se vendaient sur des listes de papier : certains billets ont servi deux fois, d'autres catégories ont été vendues au-delà de ce qui était prévu, et le calcul des recettes demandait une soirée entière.

Vous êtes l'équipe de développement. Le logiciel vend et contrôle les billets, respecte les quotas de chaque catégorie, gère les annulations et prépare le rapport des recettes. Le client est exigeant et il changera d'avis : concevez dès le départ pour que votre code puisse évoluer.

2. Le vocabulaire du métier

Terme	Sens pour le client
Billet	Le droit d'entrer au festival, avec un numéro unique, un prix et une zone.
Catégorie	Le groupe auquel appartient un billet : Standard, VIP, Étudiant ou Pass.
Quota	Le nombre maximal de billets qu'on peut vendre dans une catégorie.
Capacité totale	Le nombre maximal de billets vendus, toutes catégories confondues.
Prix de base	Le prix d'un billet avant ses suppléments ou ses réductions, en millimes (1 dinar = 1 000 millimes).
Zone	L'endroit du site où le billet donne accès (par exemple Fosse, Gradins ou VIP).
Panier	Un ensemble de billets qu'un spectateur veut acheter en une fois.
Justificatif	Le numéro de carte étudiante, exigé pour vendre un billet étudiant.
Backstage	Les coulisses des artistes, accessibles avec certains billets VIP.
Valide ou utilisé	L'état d'un billet : valide tant qu'il n'a pas servi à l'entrée, utilisé ensuite.
Recette	La somme d'argent que rapportent les billets vendus, moins les billets remboursés.

3. Les règles du métier

Ces règles sont celles du client. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

N°	Règle	Apparaît en
R1	Un billet a un **numéro non vide** et un **prix de base strictement positif**, en millimes (entiers).	v0.2
R2	Un billet est **valide** jusqu'à son utilisation ; on ne peut l'utiliser **qu'une seule fois**, et il ne redevient jamais valide.	v0.2
R3	Chaque type de billet a **son propre prix** (tableau ci-dessous), calculé en entiers.	v0.3
R4	Le **tarif étudiant** est inférieur de **30 %** au prix de base, **sans nombre à virgule**.	v0.3
R5	Un billet étudiant exige un **justificatif** : sans justificatif, il ne peut pas être vendu.	v0.3
R6	Un **pass festival** vaut trois entrées (une par soirée) : il devient « utilisé » à la **troisième** seulement.	v0.3
R7	À la billetterie, un **numéro de billet est unique** et la **capacité totale** ne peut pas être dépassée.	v0.3
R8	Chaque catégorie peut avoir un **quota** : le nombre de billets vendus de cette catégorie ne le dépasse **jamais**. La somme des quotas ne dépasse pas la capacité totale. Sans quota, seule la capacité totale limite.	v1.0
R9	Une vente est vérifiée **dans cet ordre** : conditions du type de billet, numéro unique, quota, capacité totale. Au premier refus, **rien n'est vendu ni encaissé**.	v1.0
R10	Annuler un billet **non utilisé** le retire des ventes, **libère une place** dans sa catégorie et **rembourse** son prix. Un billet utilisé ne s'annule pas.	v1.0

Les types de billets

Type	Catégorie	Prix du billet (millimes)	Particularité
Billet (classe mère)	Général	Le prix de base	Numéro, prix de base, zone, état
Billet standard	Standard	Prix de base, plus 2000 de frais de service	—
Billet VIP	VIP	Prix de base, plus 20000 si backstage, plus 4000 par boisson	Accès backstage (oui ou non) ; de 0 à 4 boissons
Billet étudiant	Étudiant	Prix de base moins 30 %	Justificatif obligatoire pour la vente
Pass festival	Pass	Trois fois le prix de base, moins 10000	Valable trois soirées

*Utilisez des **constantes nommées** plutôt que des nombres écrits au milieu du code. Tous les montants sont des **entiers** (millimes) : pas de nombres à virgule pour l'argent, y compris pour le tarif étudiant.*

4. Version v0.1 : Cadrage

Mail du client · v0.1

De : Fares Kammoun, Billetterie, festival Rivages d'Été

Objet : Notre future billetterie

Bonjour,

Cette année, nous en finissons avec les listes de papier. Avant de coder, montrez-moi que vous avez compris mon métier : les objets que vous avez repérés, ce que chacun sait de lui-même, et un premier billet affiché à l'écran. La console me suffit.

Cordialement.

N°	Demande du client	Notion
1.1	« En tant que responsable, je veux que vous identifiiez les objets de mon métier, afin que nous parlions le même langage. »	Analyse, UML
1.2	« En tant que responsable, je veux voir un billet à l'écran, afin de vérifier que mon besoin est compris. »	Classe, constructeur, main

Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple Billet, Billetterie et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche un billet, par exemple : `B001 · Fosse · 40000 millimes`.

Indice

Un billet connaît au moins son **numéro**, son **prix de base** et sa **zone**. Les quatre types arrivent en v0.3 : ne les devancez pas.

5. Version v0.2 : Objets fiables

Mail du client · v0.2

De : Fares Kammoun

Objet : Votre première version : des billets qui trichent

Bonjour,

Merci pour votre première version. Mes testeurs ont trouvé un billet à **prix négatif**, un billet sans numéro, et surtout **un même billet qui a servi à deux personnes** !

Un billet doit rester cohérent : un numéro, un prix strictement positif, et un état « valide » qui devient « utilisé » une fois, une seule. Personne ne doit pouvoir remettre l'état à « valide » à la main.

Je veux aussi calculer le prix d'un billet et le lire en une ligne.

Cordialement.

N°	Demande du client	Notion
2.1	« Je veux qu'un billet ait toujours un numéro et un prix de base strictement positif, afin d'éviter des billets absurdes. » (R1)	Encapsulation, validation
2.2	« Je veux qu'un billet ne puisse servir qu'une fois, afin d'empêcher la fraude à l'entrée. » (R2)	État d'un objet
2.3	« Je veux savoir si un billet est encore valide, afin de contrôler les entrées. »	Méthode booléenne
2.4	« Je veux connaître le prix d'un billet en millimes entiers, afin de calculer mes recettes. »	Méthode métier
2.5	« Je veux créer un billet avec son numéro et son prix (la zone est facultative), afin de gagner du temps. »	Constructeurs surchargés
2.6	« Je veux lire un billet en une ligne, avec son état, afin de contrôler vite. »	toString()

Vous livrez

- Tous les attributs **privés**, et **aucun setter pour l'état** : seule la méthode d'utilisation le modifie.
- Deux constructeurs (avec ou sans zone) qui **valident** le numéro et le prix reçus.
- Une **Demo** de trois billets au moins, avec **des refus** : numéro vide, prix négatif, double utilisation.

Indices

Testez les **valeurs limites** : prix de -5, 0 et 1 ; numéro vide ou fait d'espaces (`isEmpty` et `trim` aident). Validez aussi dans le constructeur : c'est la porte d'entrée la plus oubliée.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Numéro de billet vide : « SANS-NUMERO » utilisé.
Prix de base invalide (-5) : 1 utilisé.
B001 · Gradins · 40000 millimes · valide
B002 · Fosse · 40000 millimes · valide
B003 · Fosse · 1 millimes · valide
Prix de B001 : 40000 millimes
B001 utilisé ? true
B001 · Gradins · 40000 millimes · utilisé
  Billet B001 déjà utilisé : entrée refusée.
B001 utilisé une seconde fois ? false
B001 valide ? false · B002 valide ? true
```

6. Version v0.3 : La famille

Mail du client · v0.3

De : Fares Kammoun

Objet : Quatre types de billets, et ce n'est pas fini

Bonjour,

Excellent travail : plus aucune fraude à l'entrée. Passons à la vraie billetterie. Nous vendons quatre types de billets : **standard**, **VIP** (backstage et boissons), **étudiant** (sur justificatif, à moins 30 %) et **pass festival** (trois soirées). Chacun a son prix (voir le tableau).

Je veux une **billetterie** qui vend des billets de tous types mélangés, les affiche d'un coup, connaît la recette, et calcule le total d'un **panier**.

Et une chose importante : l'an prochain, j'ajouterai peut-être un **billet Groupe**. **Je ne veux pas que vous réécriviez votre programme à chaque nouveau type.**

Cordialement.

N°	Demande du client	Notion
3.1	« Je veux quatre types de billets avec leurs particularités (backstage, boissons, justificatif, soirées), afin de décrire mon offre. »	Héritage, super(...)
3.2	« Je veux que chaque type calcule son propre prix et annonce sa catégorie, afin d'appliquer mes tarifs. » (R3, R4)	Redéfinition, @Override
3.3	« Je veux qu'un billet étudiant sans justificatif ne se vende pas, et qu'un pass serve trois fois, afin de respecter mes règles. » (R5, R6)	Redéfinition d'une méthode métier
3.4	« Je veux une billetterie qui vend et affiche des billets de tous types, avec des numéros uniques et une capacité totale, afin de suivre mes ventes. » (R7)	Tableau ou liste du type commun
3.5	« Je veux le total d'un panier de billets et la recette des billets vendus, afin de facturer mes clients. »	Polymorphisme, une seule boucle
3.6	« Je veux ajouter un nouveau type de billet plus tard sans modifier la billetterie, afin que mon logiciel évolue. »	Preuve du polymorphisme

Vous livrez

- Une **classe mère** concrète qui porte ce qui est commun, et **quatre classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Une **Billetterie** qui contient des billets de types différents, et **une seule boucle** pour les afficher, calculer un panier et la recette.
- La preuve que la billetterie n'a pas besoin d'être modifiée pour accueillir un nouveau type.

Indices

Si vous écrivez **instanceof** dans une boucle de la billetterie, c'est que le polymorphisme manque : quelles méthodes chaque type devrait-il redéfinir ? Le pass doit se souvenir de ses entrées restantes : qui protège ce compteur ? Une méthode redéfinie peut appeler `super.utiliser()` quand elle n'a plus rien à ajouter. Pour comparer deux numéros (du texte), utilisez `equals`, jamais `==`.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Vendu : B001 · Standard · 42000 millimes
Vendu : B002 · VIP · 108000 millimes
Vendu : B003 · Étudiant · 28000 millimes
Vendu : P001 · Pass · 110000 millimes
Refusé : B001 · Standard → Numéro déjà vendu.
Refusé : B004 · Étudiant → Justificatif étudiant manquant.
Festival Rivages d'Été (4/10 billets)
  Standard B001 · Fosse · 42000 millimes · valide
  VIP B002 · VIP · 108000 millimes · valide · backstage · 2 boissons
  Étudiant B003 · Fosse · 28000 millimes · valide · carte ETU-2041
  Pass P001 · Fosse · 110000 millimes · valide · 3 soirées restantes
Total du panier de 3 billets : 178000 millimes
--- Entrées ---
Entrée acceptée : B001 · utilisé
  Billet B001 déjà utilisé : entrée refusée.
Entrée acceptée : P001 · valide
Entrée acceptée : P001 · valide
Entrée acceptée : P001 · utilisé
```

*Remarquez le pass : il reste « valide » après ses deux premières entrées et devient « utilisé » à la troisième. Le prix de chaque billet vient de **sa** méthode redéfinie, pas de la billetterie.*

7. Version v1.0 : Livraison

Mail du client · v1.0

De : Fares Kammoun

Objet : Dernière étape avant l'ouverture des ventes

Bonjour,

Votre billetterie fonctionne bien. Dernière étape avant l'ouverture des ventes. Mon cauchemar, c'est la **sur-réservation** : chaque catégorie a un **quota**, et je ne veux jamais vendre un billet de trop. Quand une vente est refusée, je veux savoir pourquoi, et un refus ne doit rien vendre ni rien encaisser.

Un spectateur peut aussi **annuler** : son billet est remboursé et **la place est libérée** pour quelqu'un d'autre, sauf si le billet a déjà servi. Et je veux le **rapport des recettes par catégorie**. Attention : le tarif étudiant à moins 30 % doit rester exact au millime, même pour un prix de base comme 45550.

Un menu simple pour mes agents, et un fichier que je puisse lancer moi-même.

Cordialement.

N°	Demande du client	Notion
4.1	« Je veux vendre un billet par une seule porte, qui vérifie les conditions dans l'ordre et ne change rien en cas de refus, afin d'éviter les erreurs. » (R9)	Assemblage
4.2	« Je veux un quota par catégorie et le refus de toute sur-réservation, afin de ne jamais vendre un billet de trop. » (R8)	Défi métier, encapsulation
4.3	« Je veux annuler un billet non utilisé, le rembourser et libérer sa place, afin de la revendre. » (R10)	État d'un objet
4.4	« Je veux le rapport des recettes par catégorie, avec le nombre de billets vendus, afin de piloter le festival. »	Calcul en entiers, une seule boucle
4.5	« Je veux un menu console, afin que mes agents vendent sans toucher au code. »	Menu, boucle de saisie
4.6	« Je veux un fichier que je lance moi-même, afin de l'installer sur mon poste. »	Fichier .jar

Le défi métier : les quotas et la sur-réservation

Une vente passe par **quatre contrôles, toujours dans le même ordre** :

1. le type de billet accepte la vente (par exemple : justificatif étudiant présent) ;

2. le numéro n'est pas déjà vendu ;

3. le quota de la catégorie n'est pas atteint (sans quota, seule la capacité limite) ;

4. la capacité totale n'est pas atteinte.

Au premier refus, rien n'est vendu ni encaissé. Une **annulation** retire le billet des ventes : la place redevient disponible, et le quota se vérifie sur le **vrai** nombre de billets vendus.

La billetterie **ne demande jamais** « quel est ton type ? » : chaque billet connaît sa catégorie, son prix et ses conditions de vente.

Menu console attendu (exemple) :

```
=== FESTIVAL RIVAGES D'ÉTÉ : BILLETTERIE ===
1. Afficher les billets vendus
2. Vendre un billet
3. Annuler un billet
4. Contrôler une entrée
5. Rapport des recettes
0. Quitter
Votre choix (0 à 5) :
```

Exemple de sortie d'un scénario de ventes (la vôtre peut différer) :

```
Quota refusé : la somme des quotas (12) dépasserait la capacité (8).
Vendu : S01 · Standard · 42000 millimes
Vendu : S02 · Standard · 42000 millimes
Refusé : S03 · Standard → Quota atteint : Standard 2/2.
Refusé : S01 · Standard → Numéro déjà vendu.
Vendu : V01 · VIP · 108000 millimes
Refusé : V02 · VIP → Quota atteint : VIP 1/1.
Vendu : E01 · Étudiant · 28000 millimes
Refusé : E02 · Étudiant → Justificatif étudiant manquant.
Vendu : P01 · Pass · 110000 millimes
--- Entrée et annulations ---
Entrée acceptée : S01 · utilisé
Annulation impossible : billet S01 déjà utilisé.
Annulé : S02 · Standard · remboursé 42000 millimes
Vendu : S03 · Standard · 42000 millimes
Rapport des recettes · Festival Rivages d'Été
  Standard : 2/2 vendus · 84000 millimes
  VIP : 1/1 vendus · 108000 millimes
  Étudiant : 1/2 vendus · 28000 millimes
  Pass : 1/2 vendus · 110000 millimes
  Total : 5 billets · 330000 millimes
```

*Remarquez la quatrième vente : le quota Standard est plein, mais le motif affiché est le numéro, parce que ce contrôle vient **avant** le quota. Après l'annulation de S02, la place est libre et S03 passe.*

Indices

Comptez les billets vendus d'une catégorie **en parcourant les ventes**, plutôt qu'avec un compteur séparé qu'on oublie de corriger à l'annulation. Quand vous retirez un billet du tableau, **décalez les suivants** : une case vide ferait planter vos boucles. Pour le tarif étudiant, **multipliez avant de diviser** : `prixBase * 70 / 100` reste exact, `prixBase / 100 * 70` perd des millimes.

8. Les bonus du sujet

Badge	Défi
Abstrait	Rendez la classe mère **abstraite** : un « billet général » n'est pas à vendre. Faites de `calculerPrix` et de `getCategorie` des méthodes abstraites. Que répond Java si vous écrivez `new Billet(...)` ? Justifiez votre choix dans le rapport.
Résilient	Créez une **exception personnalisée** (par exemple `QuotaAtteintException`) levée quand une vente dépasse un quota, avec le motif dans son message, et interceptée par le menu.
Archiviste	Sauvegardez les ventes dans un fichier texte ou CSV à la fermeture (une ligne par billet : type, numéro, prix de base, zone, état) et rechargez-les au démarrage en les repassant par la méthode de vente.

9. Les pièges fréquents de ce sujet

- **Un setter sur l'état** : n'importe quelle classe pourrait remettre un billet utilisé à « valide ».
- **Le tarif étudiant avec des nombres à virgule** : `prixBase * 0.7` donne un nombre à virgule ; restez en entiers.
- **Diviser avant de multiplier** : `prixBase / 100 * 70` perd les millimes pour un prix comme 45550.
- **Un quota testé avec > au lieu de >=** : on vend alors un billet de trop.
- **Un compteur de ventes non corrigé à l'annulation** : la place libérée ne revient jamais.
- **Une case vide dans le tableau après une annulation** : décalez les billets suivants.
- **Un instanceof dans la boucle de la billetterie** : un billet Groupe obligerait à la modifier.
- **Un refus qui laisse une trace** : un billet ajouté ou une somme encaissée après un refus.

10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi l'état d'un billet n'a-t-il pas de setter ? Quelles règles sont ainsi protégées ?
- Quelle duplication de code la classe mère a-t-elle évitée entre standard, VIP, étudiant et pass ?
- Quelle est la ligne de code où le **type réel** d'un billet décide de son prix ?
- Comment votre code garantit-il qu'un quota n'est jamais dépassé, même après une annulation ?
- Que faudrait-il modifier pour ajouter un billet Groupe ? Combien de classes existantes ?

11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter un **billet Groupe** (le prix de 4 billets, quota d'une seule vente) et le voir dans le rapport sans toucher à la billetterie.
- Passer le tarif étudiant de moins 30 % à moins 25 %.
- Faire coûter 5000 millimes de plus aux billets VIP avec backstage.
- Faire valoir le pass pour deux soirées au lieu de trois.

Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

Relation	Sens
BilletStandard, BilletVIP, BilletEtudiant, PassFestival → Billet	Héritage : chaque type **est un** billet.
Billetterie → Billet (de 0 à plusieurs)	La billetterie **contient** des billets de tous types, par le type commun.
Demo, Menu → Billetterie	Les classes de test et de menu **utilisent** la billetterie.